AshvaTrace

The AI Proposes. The Code Verifies. The Code Keeps Score.

AI Builder Cup 2026 · Manufacturing: Intelligent Operations & Industrial Efficiency

Team Ashvamedh · October 2026

The Problem: Unplanned Downtime is Devouring Manufacturing

$1.4T	$124K	49→81 min	52.7%
Annual global downtime cost	Median cost per hour of downtime	MTTR increase (2019→2024)	Downtime reduction with AI (NIST)

When a CNC machine drifts off its expected power baseline, operators face a wall of sensor data. Finding the root cause takes 30-90 minutes of manual investigation — or worse, a wrong guess that sends a technician to the wrong part. Every minute of misdiagnosis is $2,000+ in lost production.

The gap: AI can propose hypotheses, but nobody trusts AI to verify the fix. You need a system where AI investigates, a human approves, and deterministic code proves the outcome.

AshvaTrace: AI Proposes, Code Verifies

A closed-loop fault investigation system where AI does the thinking and deterministic code does the proving.

1. DETECT
Real-time power/vibration/cycle-time monitoring with configurable thresholds. Alerts fire the moment a machine drifts.

2. INVESTIGATE (AI, about 10 seconds)
Gemini (Vertex AI) reads real power, vibration and part-time data, the plant's SOPs and past fixes on the same machine, and proposes 3 ranked causes with evidence for and against, a quick check and a suggested fix. A second AI pass reviews it, then code checks every number quoted.

3. APPROVE (Human)
The engineer picks the cause they believe and approves a fix. Before anything happens, the AI commits to a number: "power should settle near 1.5%".

4. VERIFY (Code)
Plain code (NOT AI) compares live readings before and after, returns PASS or FAIL, scores the AI's prediction, and works out the energy, cost and CO2 saved. The AI is then graded on an AI report card.

Why AshvaTrace is Different

AI-Proposes / Code-Verifies Loop
Every intervention is verified by deterministic code, not another AI call. This is the trust boundary that makes the system production-ready.

The AI Is Graded by Code
The AI commits to a number before the fix. Code scores it afterwards and builds a report card: fixes that passed, predictions that came true, and whether "70% sure" really means 70%.

No Made-Up Numbers
A second AI reviews the first, then code checks that every figure quoted exists in the sensor data. Last eval: 68 of 68 figures matched.

RAG over Plant SOPs
Hypotheses are grounded in the plant's actual standard operating procedures, not generic LLM knowledge. The AI cites specific SOP sections.

Shows the Money
An open problem shows the electricity it is wasting right now. After a PASS, code shows the kW, dollars a day and CO2 the fix saves, with its assumptions on screen.

Honesty by Design
If the AI call fails, the system says so. It never substitutes canned content. Failed investigations are logged and the incident reverts to "open".

Google Cloud Architecture

Gemini on Vertex AI
Function calling over live sensor tools, a two-pass propose/critique reasoning chain, structured JSON output, and Vision AI for a separate tool-wear check. Real token cost is measured per incident.

BigQuery
Persistent storage for machines, incidents, alerts, SOPs and audit logs. Feeds the AI's memory of past fixes and its report card.

Cloud Run
Full-stack app (React frontend + Express backend) in a single container, pinned to one instance because state lives in memory.

Pub/Sub
Event-driven alerting: threshold crossings publish to a Pub/Sub topic, enabling downstream consumers (dashboards, notifications, CMMS integration).

Cloud Build
CI/CD pipeline: git push → Docker build → Cloud Run deploy. Zero manual deployment steps.

React + Vite + TypeScript
Frontend with real-time SSE alerts, interactive charts, drag-and-drop dashboard, and investigation trace visualization.

Measured Results (not claims)

100%	68 / 68	~10 s	~$0.0004
Correct cause in top 3 (4 of 4 faults)	Quoted figures match sensor data	Time for Ask AI (was 17-48 s)	AI cost per investigation

We replayed 4 faults with known root causes through the real pipeline and scored them with plain code, no AI judge. Two were deliberate false alarms (sensor drift, harmless tool change) to check the AI does not cry wolf. Top-ranked accuracy varied between 50% and 100% across runs, so we report the range. Anyone can re-run it: npm run eval.

Worked example from the live app: a fix that removes 4.7 kW of waste saves about $9 a day in electricity and 1,609 kg of CO2 a month (16 running hours a day, $0.12/kWh, 0.71 kg CO2/kWh), calculated by code from the before and after readings.

Machine data is simulated; Gemini, BigQuery and Cloud Run are real. Industry figures on slide 2: ABB 2023 ($124K/hr downtime), Siemens 2024 (81 min MTTR), NIST 2023 (52.7% reduction with AI).

Live Demo: 3-Minute Walkthrough

0:00 Monitoring: five simulated machines. One needs action; the banner shows what it is wasting in electricity right now.

0:25 Open the problem, press Ask AI. Progress streams live; three ranked causes appear in about 10 seconds.

0:55 Read one cause: evidence for and against, a quick check, a suggested fix, and the badge "code checked: all numbers match".

1:25 Open "How the AI got here": the live readings it pulled, in plain words.

1:45 Fix tab: the suggested fix is pre-filled. Preview it: the AI commits to a number.

2:10 Verify: code returns PASS with a before and after table, scores the AI's prediction, and shows $ per day and CO2 saved.

2:40 Analytics: the AI report card. Close on: the AI proposes, the code decides, and the code keeps score.

Beyond the Hackathon

Phase 1 (Now)
Live on Cloud Run: real Gemini investigation, code-verified outcomes, AI report card, BigQuery persistence, cost and savings, accuracy eval harness.

Phase 2 (3 months)
Connect to real PLC/CNC data via OPC-UA or MQTT. Replace synthetic data with live sensor streams. Add Dataflow for real-time processing.

Phase 3 (6 months)
Multi-plant deployment. CMMS integration (SAP PM, Maximo). Predictive maintenance: warn before a limit is crossed, trained on the incident history.

Phase 4 (12 months)
Use the AI report card to adjust the AI confidence automatically. Digital twin for what-if fixes. Fleet-level analytics across 100+ machines. Code always keeps the last word on verification.

AshvaTrace is live at: https://ashvatrace-206380300251.us-central1.run.app
GitHub: github.com/akanthed/AshvaTrace